topic_2

Theory, Edge Cases & Code Examples

- ✓ Iterator Management and Iteration Safety
- ✓ Type-Safe Null Pointers and Overload Resolution
- ✓ Scoped Enumerations and Type Safety
- ✓ Range-Based For Loop with Temporary Containers
- ✓ Range-Based For Loop Modifying Container Size

→ *Swipe to learn* →

topic_2

Theory

📌 1. Range-Based For Loops - Iterator Management and Iteration Safety (1/7)

Range-Based For Overview:

Range-based for loops (introduced in C++11) provide syntactic sugar over traditional iterator-based iteration, automatically managing `begin()` and `end()` iterators while supporting any type providing these functions. The compiler transforms range-based syntax into traditional iterator code, ensuring zero runtime overhead while dramatically improving readability and reducing iterator-related errors.

Syntax and Desugaring:

```
// Range-based for syntax
for (declaration : range_expression) {
    // loop body
}

// Desugared to (simplified):
{
    auto&& __range = range_expression;
    auto __begin = std::begin(__range);
    auto __end = std::end(__range);
    for (; __begin != __end; ++__begin) {
        declaration = *__begin;
        // loop body
    }
}
```

topic_2

Theory

📌 1. Range-Based For Loops - Iterator Management and Iteration Safety (2/7)

Supported Range Types:

Range Type	Requirements	Example	Notes
STL Containers	Member `begin()`/`end()`	`std::vector`, `std::list`, `std::map`	All standard containers
C-style Arrays	Compile-time known size	`int arr[5]`	Uses `std::begin`/`std::end`
std::array	Member `begin()`/`end()`	`std::array<int, 5>`	Fixed-size container
User-defined Types	Member or free `begin()`/`end()`	Custom range classes	Iterator-like objects
std::initializer_list	Temporary brace-init	`{1, 2, 3}`	Supports inline lists
std::string	Member `begin()`/`end()`	`std::string str`	Character iteration

Type Deduction Patterns:

Pattern	Declaration	Element Type	Behavior	Use Case
Copy	`auto x`	Value (copy)	Independent from container	Local computation on copies
Const Reference	`const auto& x`	Const lvalue reference	Read-only, no copies	Default for iteration
Mutable Reference	`auto& x`	Lvalue reference	Modify original elements	In-place modification
Universal Reference	`auto&& x`	Universal reference	Binds to proxies/rvalues	Generic code, proxy types
Explicit Type	`int x`	Specified type	May copy or convert	When type known and desired

```
std::vector<std::string> names = {"Alice", "Bob", "Charlie"};

// Pattern 1: Copy (expensive for strings!)
// ...continues on next slide
```

topic_2

Theory

📌 1. Range-Based For Loops - Iterator Management and Iteration Safety (3/7)

[Code continues across slides]

```
// ...continued from previous slide
for (auto name : names) {
    // name is std::string (copy), modifications don't affect names
    name += " (copy)";  // Modifies copy only
}

// Pattern 2: Const Reference (optimal for read-only)
for (const auto& name : names) {
    // name is const std::string&, no copies, read-only
    std::cout << name << "\n";  // ✅ Efficient
}

// Pattern 3: Mutable Reference (modify originals)
for (auto& name : names) {
    // name is std::string&, modifications affect names
    name += " (modified)";  // ✅ Modifies original
}

// Pattern 4: Universal Reference (for proxy types like vector<bool>)
std::vector<bool> flags{true, false, true};
for (auto&& flag : flags) {
    // flag binds to proxy reference type
    flag = !flag;  // ✅ Works correctly
}
```

topic_2

Theory

📌 1. Range-Based For Loops - Iterator Management and Iteration Safety (4/7)

Container-Specific Behaviors:

Container	Element Access	Iterator Type	Special Considerations
vector	Direct reference	Random access	May reallocate on modification
list	Reference via node	Bidirectional	No reallocation, stable references
map	`pair<const Key, Value>&`	Bidirectional	Key is always const
set	`const T&`	Bidirectional	Elements always const (ordering)
unordered_map	`pair<const Key, Value>&`	Forward	May rehash on modification
vector\<bool>	Proxy reference	Random access	⚠️ Special case: use `auto&&`

Map Iteration (Pairs):

```
std::map<int, std::string> ages = {{1, "Alice"}, {2, "Bob"}};

// ✅ C++11 style: access via pair members
for (const auto& pair : ages) {
    std::cout << pair.first << ": " << pair.second << "\n";
    // pair.first is const int& (key)
    // pair.second is std::string& (value)
}

// ✅ Modify values (keys always const)
for (auto& pair : ages) {
    pair.second += " (modified)";  // OK: value is mutable
    // pair.first = 10;  // ❌ Error: key is const
}

// Note: C++17 introduces structured bindings for cleaner syntax:
// for (const auto& [key, value] : ages) { ... }  // Not C++11
```

topic_2

Theory

📌 1. Range-Based For Loops - Iterator Management and Iteration Safety (5/7)

Iterator Invalidation Awareness:

Operation	vector	list	map/set	unordered_map/set
Insert	Invalidates at/after insert	None	None	All if rehash
Erase	Invalidates at/after erase	Erased only	Erased only	Erased only
push_back	All if realloc, else end	None	N/A	N/A
Iteration + Modification	❌ Dangerous	⚠️ Use erase return	⚠️ Use erase return	❌ Dangerous if rehash

```
std::vector<int> vec = {1, 2, 3, 4, 5};

// ❌ WRONG: Undefined behavior (iterator invalidation)
for (auto x : vec) {
    if (x % 2 == 0) {
        vec.push_back(x * 10);  // Reallocates, invalidates iterators!
    }
}

// ✅ CORRECT: Index-based when modifying size
for (size_t i = 0; i < vec.size(); ++i) {
    if (vec[i] % 2 == 0) {
        vec.push_back(vec[i] * 10);  // Safe with index
    }
}

// ✅ CORRECT: Two-phase (collect, then modify)
std::vector<int> to_add;
for (const auto& x : vec) {
    if (x % 2 == 0) {
        to_add.push_back(x * 10);
    }
}
vec.insert(vec.end(), to_add.begin(), to_add.end());
```

topic_2

Theory

📌 1. Range-Based For Loops - Iterator Management and Iteration Safety (6/7)

Temporary Container Lifetime Pitfall:

Scenario	Code	Behavior	Reason
Named container	`auto v = get(); for (auto& x : v)`	✅ Safe	Container lifetime extends loop
Temporary rvalue	`for (auto& x : get())`	❌ Undefined Behavior	Temporary destroyed after init
Temporary + copy	`for (auto x : get())`	✅ Safe but inefficient	Copies all elements

```
std::vector<int> get_data() {
    return {1, 2, 3, 4, 5};
}

// ❌ WRONG: Dangling references
for (const auto& x : get_data()) {
    // Temporary vector destroyed after begin()/end() called
    // x is dangling reference to freed memory
    std::cout << x;  // Undefined behavior
}

// ✅ CORRECT: Store in named variable
auto data = get_data();
for (const auto& x : data) {
    // data outlives the loop
    std::cout << x;  // Safe
}
```

topic_2

Theory

📌 1. Range-Based For Loops - Iterator Management and Iteration Safety (7/7)

Performance Implications:

```
struct LargeObject {
    char data[1024];  // 1 KB per object
};

std::vector<LargeObject> objects(1000);

// ❌ DISASTER: Copies 1000 × 1KB = 1 MB
for (auto obj : objects) {
    process(obj);  // Processes copy
}
// Measured time: ~15ms (dominated by memcpy)

// ✅ OPTIMAL: Zero copies
for (const auto& obj : objects) {
    process(obj);  // Processes original
}
// Measured time: ~0.5ms (pure processing)

// Speedup: 30x faster!
```

---

topic_2

Theory

📌 2. nullptr - Type-Safe Null Pointers and Overload Resolution (1/5)

nullptr Overview:

The `nullptr` keyword introduces a dedicated null pointer literal with type `std::nullptr_t` that converts to any pointer type but not to integral types. This solves the longstanding ambiguity where `NULL` (typically defined as `0`) causes overload resolution issues when both pointer and integer overloads exist.

Historical Problem with NULL:

Literal	Type	Converts to Pointer?	Converts to Int?	Overload Preference
`0`	`int`	✅ Yes (null pointer constant)	✅ Yes (is int)	Integer overloads
`NULL`	Implementation-defined (usually `int` or `long`)	✅ Yes (null pointer constant)	✅ Yes	Integer overloads
`nullptr`	`std::nullptr_t`	✅ Yes (any pointer type)	❌ No	Pointer overloads

```
void func(int x) {
    std::cout << "int version: " << x << "\n";
}

void func(char* ptr) {
    std::cout << "pointer version: " << (ptr ? "valid" : "null") << "\n";
}

// Overload resolution comparison
func(0);        // ✅ Calls int version (0 is int)
func(NULL);     // ⚠️  Typically calls int version (NULL usually 0)
func(nullptr);  // ✅ Calls pointer version (nullptr converts to pointer)
```

topic_2

Theory

📌 2. nullptr - Type-Safe Null Pointers and Overload Resolution (2/5)

nullptr Type Properties:

Property	Value	Explanation
Type	`std::nullptr_t`	Unique type defined in `<cstddef>`
Size	Implementation-defined	Typically same as pointer size (4 or 8 bytes)
Value	Single value: `nullptr`	Only one possible value
Pointer conversion	Implicit to any pointer type	`int*`, `char*`, `void*`, `T*`
Integer conversion	None (explicit conversion required)	Cannot convert to `int`, `bool` implicitly
Boolean conversion	Explicit `false`	`static_cast<bool>(nullptr) → false`
Comparison	Equality with pointers and `nullptr`	`ptr == nullptr`, `nullptr == nullptr`

Type Safety Comparison:

```
// NULL problems (legacy)
int x = NULL;        // ✅ Compiles (NULL is 0)
bool b = NULL;       // ✅ Compiles (NULL converts to false)
if (NULL) { }        // ✅ Compiles (NULL is 0, falsy)

// nullptr type safety
int y = nullptr;     // ❌ Compile error: no conversion to int
bool c = nullptr;    // ❌ Compile error: no implicit bool conversion
if (nullptr) { }     // ❌ Compile error: no implicit bool conversion

// Pointer assignments
void* p1 = NULL;      // ✅ OK
void* p2 = nullptr;   // ✅ OK
int* p3 = nullptr;    // ✅ OK: converts to any pointer type
```

topic_2

Theory

📌 2. nullptr - Type-Safe Null Pointers and Overload Resolution (3/5)

Overload Resolution Examples:

```
// Example 1: Pointer vs Integer
void process(int value) {
    std::cout << "Processing int: " << value << "\n";
}

void process(void* ptr) {
    std::cout << "Processing pointer\n";
}

process(42);       // ✅ int overload
process(NULL);     // ⚠️  int overload (NULL is 0)
process(nullptr);  // ✅ pointer overload

// Example 2: Multiple Pointer Types
void handle(int* ptr) {
    std::cout << "int pointer\n";
}

void handle(char* ptr) {
    std::cout << "char pointer\n";
}

// handle(nullptr);  // ❌ Ambiguous: nullptr converts to both

// Example 3: std::nullptr_t Overload
void log(int error_code) {
    std::cout << "Error code: " << error_code << "\n";
}

void log(std::nullptr_t) {
    std::cout << "Null pointer logged\n";
}

log(0);        // ✅ int overload
log(nullptr);  // ✅ nullptr_t overload
```

topic_2

Theory

📌 2. nullptr - Type-Safe Null Pointers and Overload Resolution (4/5)

Smart Pointer Integration:

```
#include <memory>

// Initialization
std::shared_ptr<int> sp1 = nullptr;        // ✅ Empty smart pointer
std::unique_ptr<int> up1(nullptr);         // ✅ Empty smart pointer
std::shared_ptr<int> sp2 = std::make_shared<int>(42);

// Comparison
if (sp1 == nullptr) {
    std::cout << "sp1 is empty\n";  // ✅ Idiomatic null check
}

if (!sp2) {
    std::cout << "sp2 is empty\n";  // Also valid, but nullptr more explicit
}

// Assignment (releases ownership)
sp2 = nullptr;  // ✅ Releases owned object, becomes empty
```

Function Return Types:

```
// Returning nullptr
int* find_value(const std::vector<int>& vec, int target) {
    for (auto& val : vec) {
        if (val == target) {
            return &val;
        }
    }
    return nullptr;  // ✅ Type-safe null return
}

// Caller code
auto* ptr = find_value(numbers, 42);
if (ptr != nullptr) {
    std::cout << "Found: " << *ptr << "\n";
} else {
    std::cout << "Not found\n";
}
```

topic_2

Theory

📌 2. nullptr - Type-Safe Null Pointers and Overload Resolution (5/5)

Template Deduction:

```
template<typename T>
void func(T param) {
    // What is T?
}

func(0);        // T = int
func(NULL);     // T = int (or long, implementation-defined)
func(nullptr);  // T = std::nullptr_t

// More useful: perfect forwarding
template<typename T>
void forward_to_process(T&& ptr) {
    process(std::forward<T>(ptr));  // Forwards with correct type
}

forward_to_process(nullptr);  // ✅ Forwards as std::nullptr_t
forward_to_process(NULL);     // ⚠️  Forwards as int
```

Best Practices:

1. **Always use nullptr for null pointers** - Never use `0` or `NULL`
2. **Explicit in comparisons** - `if (ptr == nullptr)` more readable than `if (!ptr)`
3. **Smart pointer initialization** - `std::unique_ptr<T> ptr = nullptr;`
4. **Function parameters** - Accept `std::nullptr_t` for null-specific overloads
5. **Return null from functions** - `return nullptr;` instead of `return 0;`
6. **Template code** - Essential for correct type deduction in generic code

---

topic_2

Theory

📌 3. enum class - Scoped Enumerations and Type Safety (1/8)

enum class Overview:

Strongly typed enumerations (`enum class`) address three major problems with traditional enums:

1. **Namespace pollution** - Enumerators inject into enclosing scope
2. **Implicit integer conversion** - Enables meaningless arithmetic/comparisons
3. **Implementation-defined underlying type** - Portability and size issues

enum vs enum class Comparison:

Feature	Traditional enum	enum class
Scoping	Unscoped (pollutes enclosing namespace)	Scoped (requires qualification)
Implicit int conversion	✅ Yes (can use as int)	❌ No (requires explicit cast)
Cross-enum comparison	✅ Allowed (via int conversion)	❌ Compile error
Underlying type	Implementation-defined	Explicit or default to int
Forward declaration	Requires underlying type	Requires underlying type
Switch exhaustiveness	Compiler can warn	Compiler can warn
Namespace pollution	❌ Yes (enumerators in scope)	✅ No (enumerators scoped)
Bitwise operations	✅ Works (via int conversion)	❌ Requires operator overloads

topic_2

Theory

📌 3. enum class - Scoped Enumerations and Type Safety (2/8)

Syntax and Usage:

```
// Traditional enum (C-style)
enum Color {
    Red,
    Green,
    Blue
};

Color c1 = Red;           // ✅ Unqualified access
int x = Red;              // ✅ Implicit conversion to int
if (Red == 0) { }         // ✅ Compare with int

// enum class (C++11)
enum class SafeColor {
    Red,
    Green,
    Blue
};

SafeColor c2 = SafeColor::Red;  // ✅ Qualified access required
// SafeColor c3 = Red;           // ❌ Error: Red not in scope
// int y = SafeColor::Red;       // ❌ Error: no implicit conversion
// if (SafeColor::Red == 0) { }  // ❌ Error: cannot compare with int
```

Namespace Pollution Prevention:

```
// Traditional enum: name collisions
enum Status { Active, Inactive, Pending };
enum ConnectionState { Active, Disconnected };  // ❌ Error: Active redefinition

// enum class: no collisions
enum class TaskStatus { Active, Inactive, Pending };
enum class NetworkStatus { Active, Disconnected };  // ✅ No conflict

TaskStatus task = TaskStatus::Active;
NetworkStatus net = NetworkStatus::Active;  // Different types
// if (task == net) { }  // ❌ Error: cannot compare different enum classes
```

topic_2

Theory

📌 3. enum class - Scoped Enumerations and Type Safety (3/8)

Explicit Underlying Type:

```
// Without explicit type (default to int)
enum class DefaultSize {
    Small,
    Medium,
    Large
};
static_assert(sizeof(DefaultSize) == sizeof(int), "Default is int");

// With explicit type (memory optimization)
enum class CompactStatus : uint8_t {
    OK = 0,
    Warning = 1,
    Error = 2,
    Critical = 255
};
static_assert(sizeof(CompactStatus) == 1, "Compact storage");

// With signed type (negative values)
enum class ErrorCode : int {
    Success = 0,
    FileNotFound = -1,
    PermissionDenied = -2,
    InvalidArgument = -3
};

// Large value range
enum class LargeValues : uint64_t {
    MinValue = 0,
    MaxValue = 0xFFFFFFFFFFFFFFFF
};
static_assert(sizeof(LargeValues) == 8, "64-bit storage");
```

📌 3. enum class - Scoped Enumerations and Type Safety (4/8)

Underlying Type Benefits:

Benefit	Example	Use Case
Memory optimization	`enum class Status : uint8_t`	Embedded systems, packed structs
Negative values	`enum class ErrorCode : int`	Error codes, offsets
Large ranges	`enum class ID : uint64_t`	Database IDs, large constants
ABI stability	Explicit type ensures size	Binary protocols, serialization
Forward declaration	Required for opaque types	Reduce header dependencies

Explicit Conversions:

```
enum class Level : int {
    Low = 1,
    Medium = 2,
    High = 3
};

// ✅ Explicit conversion to int
Level level = Level::Medium;
int level_value = static_cast<int>(level);  // 2
std::cout << "Level: " << level_value << "\n";

// ✅ Explicit conversion from int
int input = 3;
Level new_level = static_cast<Level>(input);

// ⚠️ No validation: dangerous with user input
int invalid = 999;
Level bad_level = static_cast<Level>(invalid);  // Valid cast, invalid value!
// Recommendation: Validate before casting
```

📌 3. enum class - Scoped Enumerations and Type Safety (5/8)

Switch Statement Exhaustiveness:

```
enum class State {
    Idle,
    Running,
    Paused,
    Stopped
};

void process_state(State s) {
    switch (s) {
        case State::Idle:    std::cout << "Idle\n"; break;
        case State::Running: std::cout << "Running\n"; break;
        case State::Paused:  std::cout << "Paused\n"; break;
        // Missing: State::Stopped
    }
    // Compiler warning: "enumeration value 'Stopped' not handled in switch"
}
```

topic_2

Theory

📌 3. enum class - Scoped Enumerations and Type Safety (6/8)

Bitwise Operations (Flags Pattern):

```
enum class Permissions : uint32_t {
    None    = 0,
    Read    = 1 << 0,  // 0x01
    Write   = 1 << 1,  // 0x02
    Execute = 1 << 2   // 0x04
};

// ❌ Bitwise operators not defined by default
// auto combined = Permissions::Read | Permissions::Write;  // Error

// ✅ Option 1: Explicit casts (verbose)
auto combined = static_cast<Permissions>(
    static_cast<uint32_t>(Permissions::Read) |
    static_cast<uint32_t>(Permissions::Write)
);

// ✅ Option 2: Overload operators
inline Permissions operator|(Permissions lhs, Permissions rhs) {
    using T = std::underlying_type_t<Permissions>;
    return static_cast<Permissions>(static_cast<T>(lhs) | static_cast<T>(rhs));
}

inline Permissions operator&(Permissions lhs, Permissions rhs) {
    using T = std::underlying_type_t<Permissions>;
    return static_cast<Permissions>(static_cast<T>(lhs) & static_cast<T>(rhs));
}

inline Permissions& operator|=(Permissions& lhs, Permissions rhs) {
    return lhs = lhs | rhs;
}

// Usage after operator overloads
auto perms = Permissions::Read | Permissions::Write;
perms |= Permissions::Execute;

if ((perms & Permissions::Read) != Permissions::None) {
    std::cout << "Has read permission\n";
}
```

topic_2

Theory

📌 3. enum class - Scoped Enumerations and Type Safety (7/8)

Type Safety in Action:

```
enum class Color { Red, Green, Blue };
enum class Size { Small, Medium, Large };
enum class Priority : int { Low = 0, Medium = 1, High = 2 };

Color c = Color::Red;
Size s = Size::Small;
Priority p = Priority::High;

// ❌ Prevented by type safety:
// if (c == s) { }                  // Error: different enum classes
// if (c == 0) { }                  // Error: no comparison with int
// int x = c;                       // Error: no implicit conversion
// Color c2 = Red;                  // Error: Red not in scope
// Size s2 = Color::Red;            // Error: type mismatch
// auto result = c + 1;             // Error: no arithmetic operators

// ✅ Type-safe comparisons within same enum
if (c == Color::Red) {
    std::cout << "Color is red\n";
}

// ✅ Type-safe switch statements
switch (p) {
    case Priority::Low:    break;
    case Priority::Medium: break;
    case Priority::High:   break;
}
```

📌 3. enum class - Scoped Enumerations and Type Safety (8/8)

Forward Declaration:

```
// Forward declaration requires underlying type
enum class Status : uint8_t;

// Can use in declarations before definition
void process_status(Status s);
class Config {
    Status current_status;  // OK: size known
};

// Later: full definition
enum class Status : uint8_t {
    Idle,
    Active,
    Error
};
```

Common Use Cases:

Use Case	Example	Why enum class
State machines	`enum class State { Idle, Running, Stopped }`	Type safety prevents invalid state transitions
Error codes	`enum class Error : int { OK = 0, NotFound = -1 }`	Scoping prevents conflicts with system error codes
Configuration options	`enum class LogLevel { Debug, Info, Warning, Error }`	No namespace pollution with common names
Bit flags	`enum class Flags : uint32_t { Read = 1, Write = 2 }`	Explicit operators make intent clear
Protocol constants	`enum class MessageType : uint8_t { Ping = 1, Data = 2 }`	Explicit size for wire protocols

topic_2

Edge Cases

📌 1. Range-Based For Loop with Temporary Containers

Using range-based for loops with temporary objects that are destroyed at the end of the full expression creates dangling references.

```
// ❌ Undefined behavior: temporary destroyed
for (const auto& x : get_vector()) {
    // x refers to elements in destroyed temporary
}

// ✅ Correct: extend temporary lifetime
auto vec = get_vector();
for (const auto& x : vec) {
    // Safe: vec outlives the loop
}
```

The temporary container returned by `get_vector()` is destroyed after the range-based for loop initialization, leaving all references dangling. This is a subtle lifetime issue that can cause crashes or data corruption. Always ensure the container outlives the loop when using references.

topic_2

Edge Cases

📌 2. Range-Based For Loop Modifying Container Size

Modifying a container's size during iteration invalidates iterators and causes undefined behavior.

```
std::vector<int> vec{1, 2, 3, 4, 5};

// ❌ Undefined behavior: modifying during iteration
for (auto x : vec) {
    if (x % 2 == 0) {
        vec.push_back(x * 10);  // Invalidates iterators!
    }
}

// ✅ Correct: iterate over indices
for (size_t i = 0; i < vec.size(); ++i) {
    if (vec[i] % 2 == 0) {
        vec.push_back(vec[i] * 10);
    }
}
```

Range-based for loops cache the end iterator, so adding elements doesn't extend the iteration. However, operations that invalidate iterators (like reallocation in `vector`) cause undefined behavior. Use index-based iteration when modifying container structure.

topic_2

Edge Cases

📌 3. nullptr with Function Overloading

The `nullptr` keyword resolves ambiguity in overload resolution that `NULL` or `0` cannot.

```
void process(int x) { std::cout << "int version\n"; }
void process(char* ptr) { std::cout << "pointer version\n"; }

process(0);        // ✅ Calls int version (0 is int)
process(NULL);     // ⚠️  Usually calls int version (NULL is typically 0)
process(nullptr);  // ✅ Calls pointer version (nullptr converts to pointer)
```

This demonstrates why `nullptr` was necessary. `NULL` being defined as `0` causes it to preferentially match integer overloads, which is rarely the intended behavior when passing null pointers. Using `nullptr` makes intent explicit and eliminates this ambiguity.

topic_2

Edge Cases

📌 4. nullptr is Not an Integer

Unlike `NULL` and `0`, `nullptr` cannot implicitly convert to integer types, preventing logic errors.

```
int x = NULL;     // ✅ Compiles (NULL is 0)
int y = nullptr;  // ❌ Compile error: cannot convert nullptr_t to int

if (nullptr) { }  // ❌ Compile error: no boolean conversion

void* ptr = nullptr;  // ✅ OK: converts to any pointer type
```

This type safety is intentional. If you're assigning or comparing with integers, you should be using `0`, not a null pointer literal. This compile-time enforcement prevents a category of logic bugs where pointer nullity is confused with numeric zero.

topic_2

Edge Cases

📌 5. enum class and Bitwise Operations

Strongly typed enums don't support bitwise operations by default, requiring explicit casts or operator overloads.

```
enum class Flags : uint32_t {
    Read = 1,
    Write = 2,
    Execute = 4
};

// ❌ Compile error: no operator| for enum class
// Flags combined = Flags::Read | Flags::Write;

// ✅ Option 1: Explicit casts
Flags combined = static_cast<Flags>(
    static_cast<uint32_t>(Flags::Read) |
    static_cast<uint32_t>(Flags::Write)
);

// ✅ Option 2: Overload operators
Flags operator|(Flags lhs, Flags rhs) {
    return static_cast<Flags>(
        static_cast<uint32_t>(lhs) | static_cast<uint32_t>(rhs)
    );
}
```

While this verbosity might seem like a disadvantage, it forces explicit handling of bit flags, making the code's intent clear. For true bit flags, consider using `std::bitset` or defining the necessary operators.

📌 6. enum class Underlying Type

Specifying the underlying type is important for ABI stability, serialization, and controlling memory usage.

```
enum class Status : uint8_t {  // Explicit 1-byte storage
    OK = 0,
    Error = 1,
    Pending = 2
};

enum class LargeValues : uint64_t {  // Explicit 8-byte storage
    FirstValue = 0,
    LargeValue = 0xFFFFFFFFFFFFFFFF
};

static_assert(sizeof(Status) == 1, "Status should be 1 byte");
static_assert(sizeof(LargeValues) == 8, "LargeValues should be 8 bytes");
```

Explicitly specifying the underlying type ensures consistent size across platforms, which is critical for binary protocols, file formats, and memory-constrained embedded systems. Without specification, the compiler chooses a type large enough to hold all values.

📌 7. Range-Based For with Different Container Types

Different containers have different reference semantics and proxy types that affect range-based for loops.

```
std::vector<int> vec{1, 2, 3};
std::vector<bool> vecBool{true, false, true};

// ✅ Works as expected
for (auto& x : vec) {
    x *= 2;  // Modifies elements
}

// ⚠️ Tricky: vector<bool> uses proxy references
for (auto x : vecBool) {
    // x is std::vector<bool>::reference (proxy type)
}

// ✅ Correct for vector<bool>
for (auto&& x : vecBool) {
    x = !x;  // Works correctly with proxy
}
```

`std::vector<bool>` is a template specialization that packs bits, returning proxy references instead of real references. Using `auto` captures the proxy type, while `auto&&` universal references work correctly with proxies.

---

topic_2

Code Examples

📌 1. Basic Range-Based For Loop Patterns

The choice between `auto`, `auto&`, and `const auto&` determines whether elements are copied, modifiable, or read-only. Use `const auto&` by default for read-only access to avoid unnecessary copies, especially with large objects. Use `auto&` when modifying elements in-place, and plain `auto` only when you need mutable local copies.

```
std::vector<int> numbers{1, 2, 3, 4, 5};

// Read-only iteration (best for const access)
for (const auto& num : numbers) {
    std::cout << num << " ";
}

// Modifying elements in-place
for (auto& num : numbers) {
    num *= 2;
}

// Creating copies (when modification needed locally only)
for (auto num : numbers) {
    num += 10;  // Modifies copy only
    std::cout << num << " ";
}
```

topic_2

Code Examples

📌 2. Range-Based For with Different Container Types

Range-based for loops work with any type providing `begin()` and `end()` functions (or for which `std::begin()` and `std::end()` are defined). This includes all standard containers, C-style arrays, and user-defined types implementing the required interface.

```
// Works with C-style arrays
int arr[] = {1, 2, 3, 4, 5};
for (int x : arr) {
    std::cout << x << " ";
}

// Works with std::map (gets pairs)
std::map<int, std::string> map{{1, "one"}, {2, "two"}};
for (const auto& pair : map) {
    std::cout << pair.first << ": " << pair.second << "\n";
}

// Works with std::array
std::array<int, 3> stdArr{10, 20, 30};
for (const auto& val : stdArr) {
    std::cout << val << " ";
}

// Works with initializer_list
for (int x : {100, 200, 300}) {
    std::cout << x << " ";
}
```

topic_2

Code Examples

📌 3. nullptr in Function Overloading

The `nullptr` keyword eliminates the historical ambiguity where `NULL` would match integer overloads instead of pointer overloads. However, when multiple pointer-type overloads exist, `nullptr` can still be ambiguous since it converts to any pointer type.

```
void handle(int value) {
    std::cout << "Processing integer: " << value << "\n";
}

void handle(const char* str) {
    std::cout << "Processing string: " << (str ? str : "null") << "\n";
}

void handle(std::shared_ptr<int> ptr) {
    std::cout << "Processing smart pointer\n";
}

int main() {
    handle(42);             // ✅ Calls int overload
    handle("hello");        // ✅ Calls const char* overload
    handle(nullptr);        // ✅ Calls const char* overload
    handle(NULL);           // ⚠️  Calls int overload (NULL is 0)

    handle(std::shared_ptr<int>()); // ✅ Calls smart pointer overload
    handle(nullptr);        // ❌ Ambiguous: converts to both raw and smart pointer
}
```

topic_2

Code Examples

📌 4. Type-Safe nullptr_t Type

The `std::nullptr_t` type can be used to create functions that specifically accept only null pointers, providing even stronger type safety. This is useful for APIs that have special handling for null cases and want to prevent accidental integer arguments.

```
void accept_nullptr(std::nullptr_t) {
    std::cout << "Received nullptr\n";
}

void accept_int(int) {
    std::cout << "Received int\n";
}

void accept_pointer(void*) {
    std::cout << "Received pointer\n";
}

int main() {
    accept_nullptr(nullptr);  // ✅ OK
    // accept_nullptr(NULL);  // ❌ Error: NULL is int, not nullptr_t
    // accept_nullptr(0);     // ❌ Error: 0 is int, not nullptr_t

    accept_pointer(nullptr);  // ✅ OK: nullptr converts to void*
    // accept_int(nullptr);   // ❌ Error: no conversion to int
}
```

topic_2

Code Examples

📌 5. Basic enum class Usage

Strongly typed enums eliminate three major sources of errors: name pollution in the enclosing scope, implicit conversions to integers, and accidental comparisons between unrelated enum types. These compile-time checks prevent logic errors without any runtime cost.

```
enum class Color {
    Red,
    Green,
    Blue
};

enum class TrafficLight {
    Red,    // ✅ No conflict with Color::Red
    Yellow,
    Green   // ✅ No conflict with Color::Green
};

Color c = Color::Red;          // ✅ Scoped access required
TrafficLight t = TrafficLight::Red;  // ✅ Distinct type

// ❌ Compile errors demonstrating safety:
// Color c2 = Red;            // Error: Red not in scope
// if (c == t) { }            // Error: cannot compare different enum classes
// int x = Color::Red;        // Error: no implicit conversion to int
// if (c == 0) { }            // Error: cannot compare enum class to int
```

topic_2

Code Examples

📌 6. enum class with Explicit Underlying Type

Specifying the underlying type provides control over memory layout, enables negative values when needed, and ensures consistent behavior across different compilers and platforms. This is essential for binary protocols, serialization, and embedded systems with strict memory constraints.

```
enum class Status : uint8_t {
    Idle = 0,
    Running = 1,
    Stopped = 2,
    Error = 255
};

enum class ErrorCode : int {
    Success = 0,
    FileNotFound = -1,
    PermissionDenied = -2,
    InvalidArgument = -3
};

// Explicit conversion to underlying type
Status status = Status::Running;
uint8_t statusValue = static_cast<uint8_t>(status);
std::cout << "Status value: " << static_cast<int>(statusValue) << "\n";

// Size is explicitly controlled
static_assert(sizeof(Status) == 1, "Status should be 1 byte");
static_assert(sizeof(ErrorCode) == sizeof(int), "ErrorCode should match int");
```

topic_2

Code Examples

📌 7. Converting Between enum class and Integers

The lack of implicit conversion is intentional and forces explicit casts, making dangerous conversions visible in code. However, `static_cast` doesn't validate that the integer value corresponds to a valid enumerator, so additional validation is needed when converting user input or external data.

```
enum class Level : int {
    Low = 1,
    Medium = 2,
    High = 3
};

// ✅ Explicit conversion from enum to int
Level level = Level::Medium;
int levelValue = static_cast<int>(level);
std::cout << "Level: " << levelValue << "\n";  // Prints: Level: 2

// ✅ Explicit conversion from int to enum
int input = 3;
Level newLevel = static_cast<Level>(input);

// ⚠️  No validation: dangerous with user input
int invalidInput = 999;
Level dangerousLevel = static_cast<Level>(invalidInput);  // No error, but invalid
```

topic_2

Code Examples

📌 8. Range-Based For with User-Defined Types

Range-based for loops work with any type that provides `begin()` and `end()` member functions returning iterator-like objects. The iterator must support `operator*` (dereference), `operator++` (increment), and `operator!=` (comparison). This allows custom ranges and views to integrate seamlessly with modern C++ syntax.

```
class IntRange {
    int start, end;
public:
    IntRange(int s, int e) : start(s), end(e) {}

    class Iterator {
        int current;
    public:
        Iterator(int val) : current(val) {}
        int operator*() const { return current; }
        Iterator& operator++() { ++current; return *this; }
        bool operator!=(const Iterator& other) const {
            return current != other.current;
        }
    };

    Iterator begin() const { return Iterator(start); }
    Iterator end() const { return Iterator(end); }
};

// ✅ Works with range-based for loop
for (int x : IntRange(1, 6)) {
    std::cout << x << " ";  // Prints: 1 2 3 4 5
}
```

What You Learned

Key Takeaways

- ✓ Iterator Management and Iteration Safety
- ✓ Type-Safe Null Pointers and Overload Resolution
- ✓ Scoped Enumerations and Type Safety
- ✓ Range-Based For Loop with Temporary Containers

→ *Swipe to learn* →